

COS10009 Week 7

Data References, Advanced Arrays and searching

Today we will cover the following topics:

7	02 Mar	Data References, Arrays, Searching
---	--------	------------------------------------

Review

Week 7: 7.1T M Concept Map

[Publish](#)[Assign to](#)[Edit](#)[⋮](#)

In your tutorial/lab groups produce a concept map in preparation for the test next week. You should cover the concepts from the lessons and tasks. First make a list of terms, then draw a diagram that organises those terms into a comprehensive overview that relates the concepts and terms to each other.

Review – Warmup Task

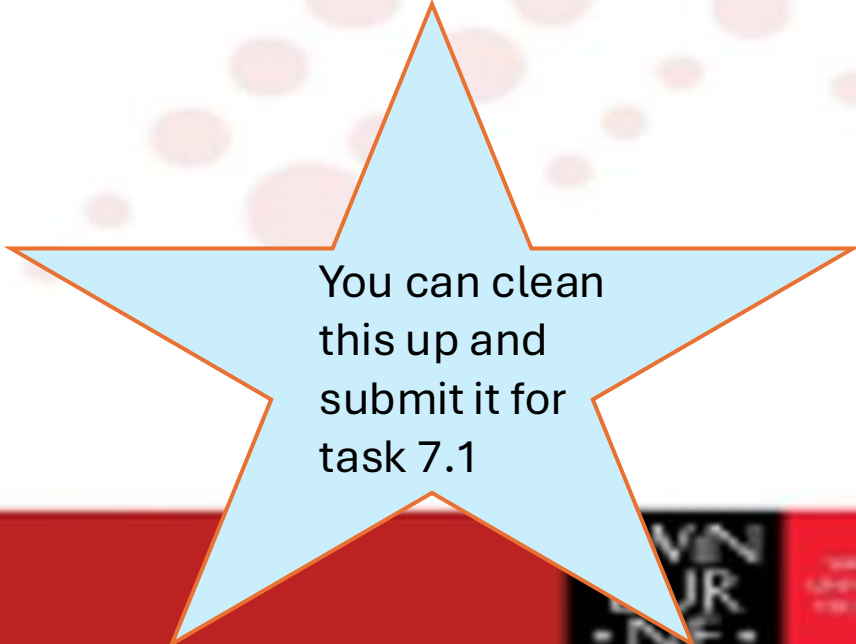
- Create a concept map – Try to link the topics and tools covered.
- On a piece of paper try to draw a map, then check with the students on your table if it is correct.
- Feel Free to use some of the words below or add your own (Python related)

Concepts - (Repetition, Functional Decomposition)

Artefacts (Byproduct) - (Program, functions, diagrams, files)

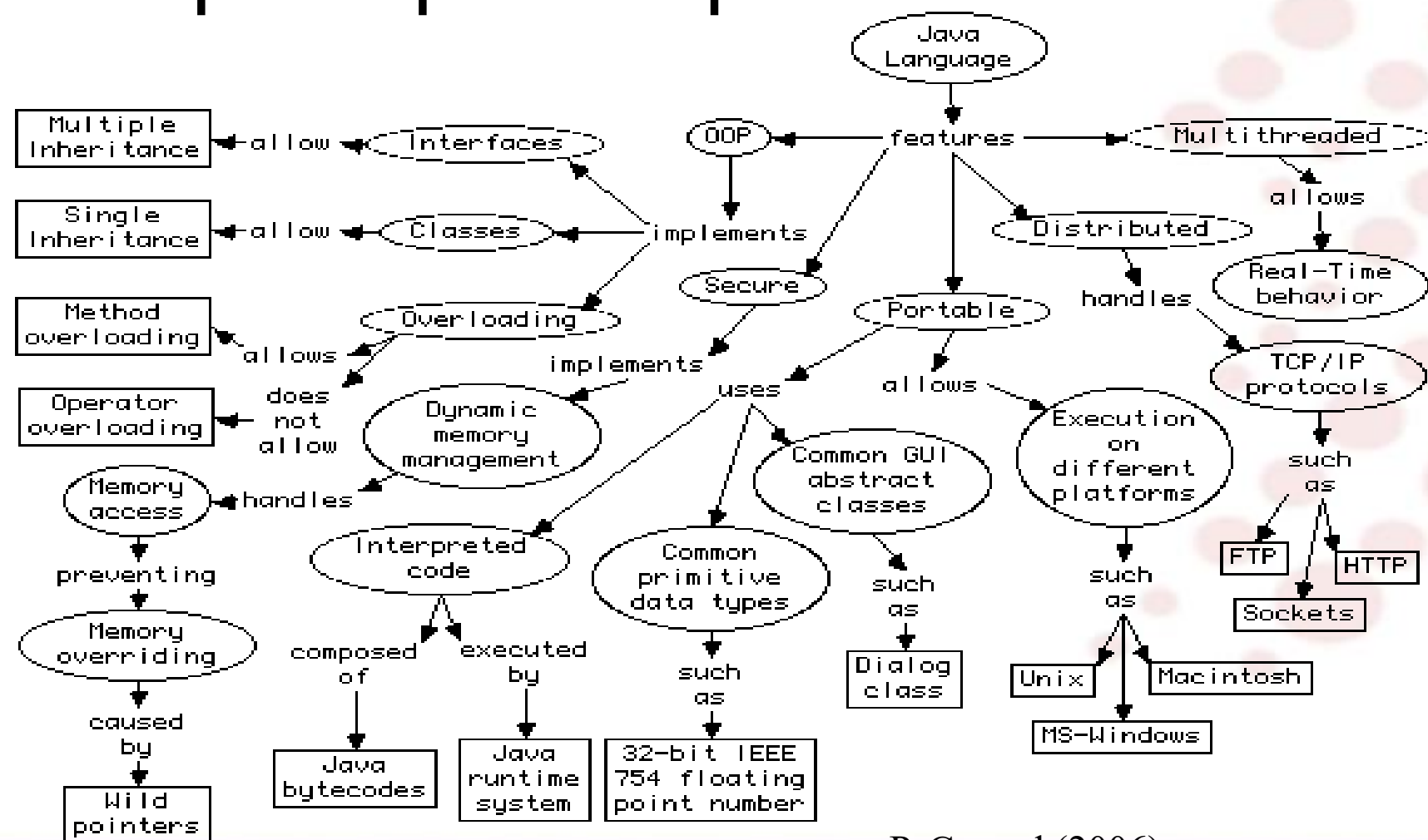
Actions - (Assignment, function call, condition evaluation)

Terminology - (Local/global variable, parameter vs argument)



You can clean
this up and
submit it for
task 7.1

Concept Map Example



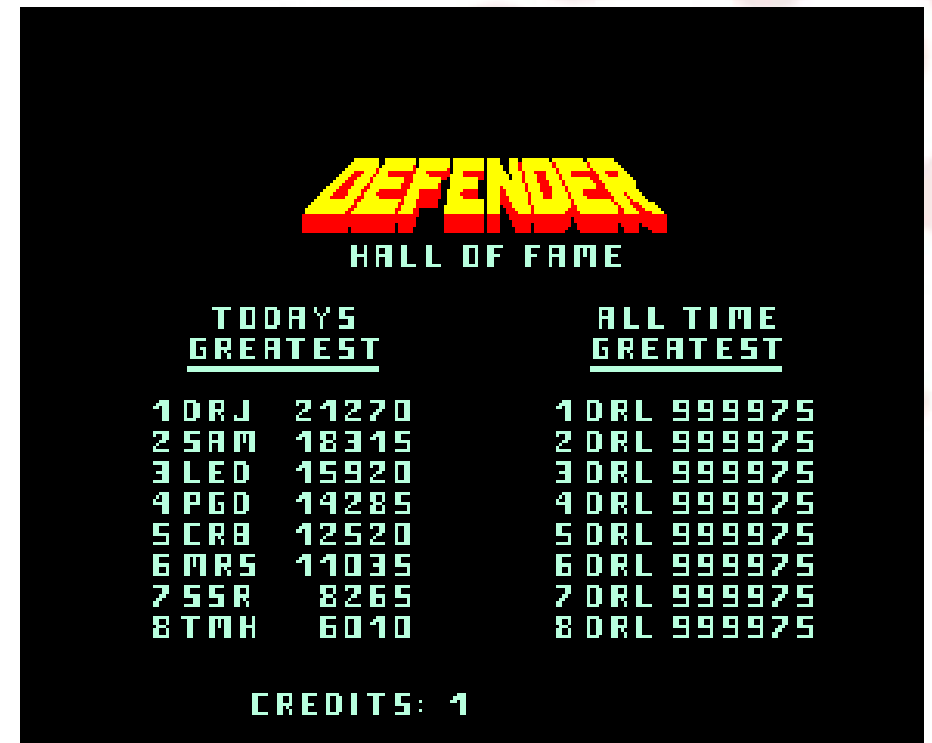
P. Conrad (2006) -

<https://www1.udel.edu/CIS/474/pconrad/06S/topics/pbl/ConceptMapping/intoToConceptMapping.htm>

Problem Scenario - Think about...

You have:

- players = ["Tom", "Sara", "Leo", "Mina"]
scores = [150, 200, 120, 180]
- Tasks:
- Find the highest score
- Print the name of the winner
- Find the rank of a given player



Advanced Arrays & Lists

- We will covered some Array Theory and then look at Lists in Python

Arrays in Concept

- What happens when we store an array of 5 strings in memory?
- We can store each of these items sequentially in the memory.

Address of element i = Base Address + (Index $i \times$ Size of Data Type)

1024	1280	1536	1792	2048
"Fred"	"Sam"	"Jill"	"John"	"Jenny"
1024 + 0	1024 + 256	1024 + 512	1024 + 768	1024 + 1024

Image Source – Ed
Lessons Week 6

Arrays in Concept

- What are the limitations with this? Can anyone figure it out?
- Keep in mind traditionally arrays have a fixed size. (Python Lists are dynamic, we will explain this later)
- We have a fixed size taken up in the memory.

Arrays in Concept – Static vs Dynamic

- A static array takes up a fixed amount of memory space, it's allocated continuous space for multiple variables of a given type.

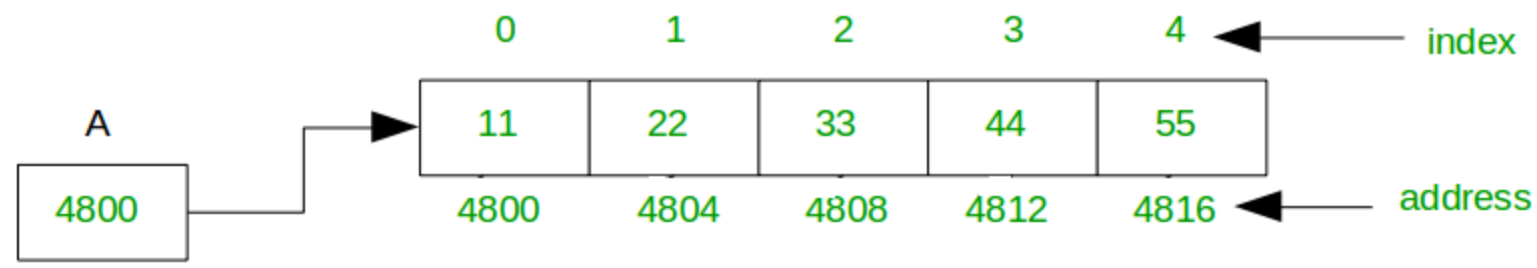


Image Source - <https://www.geeksforgeeks.org/dsa/how-do-dynamic-arrays-work/>

Arrays in Concept – Static vs Dynamic

- A dynamic array will automatically copy the array to a new larger one and update as needed.

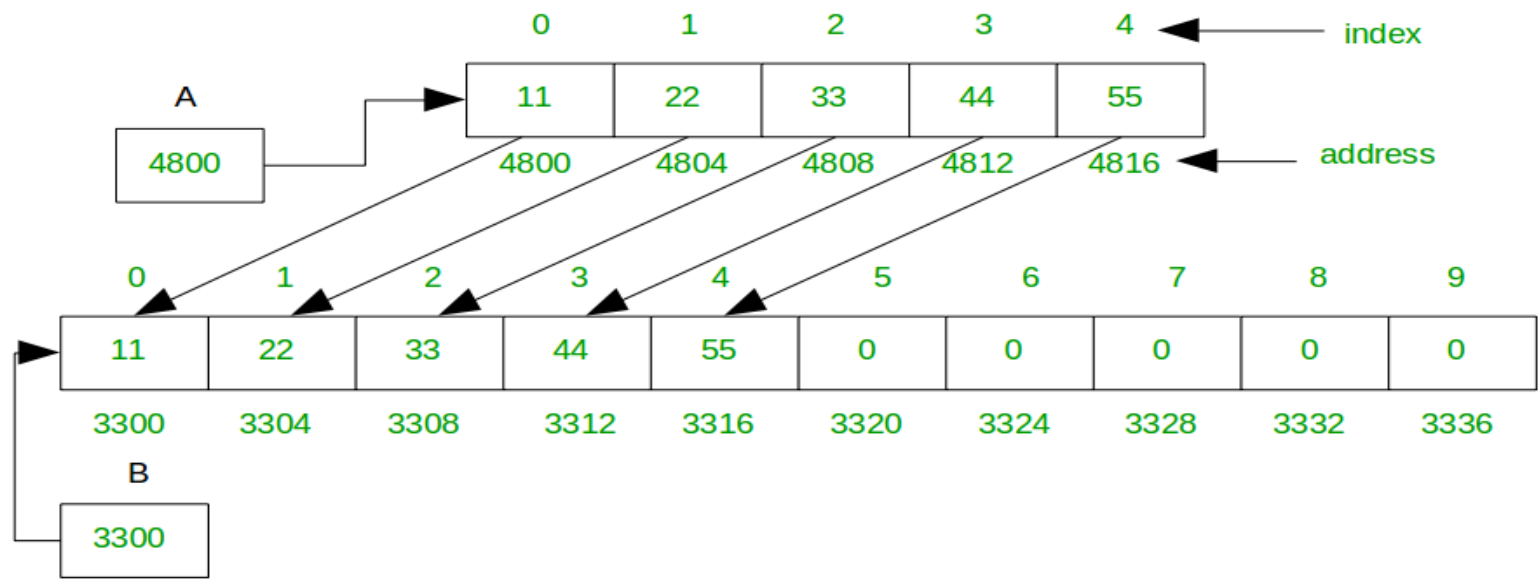


Image Source - <https://www.geeksforgeeks.org/dsa/how-do-dynamic-arrays-work/>

Arrays in Concept - Dangers

- When we access an array it's possible to go out of bounds. When we do this, it can cause our program to crash, or even worse access data from memory elsewhere and cause unexpected behaviour.

Address of element i = Base Address + (Index $i \times$ Size of Data Type)

Arrays in Concept - Dangers

- Accessing this memory address in C could return an unrelated value!

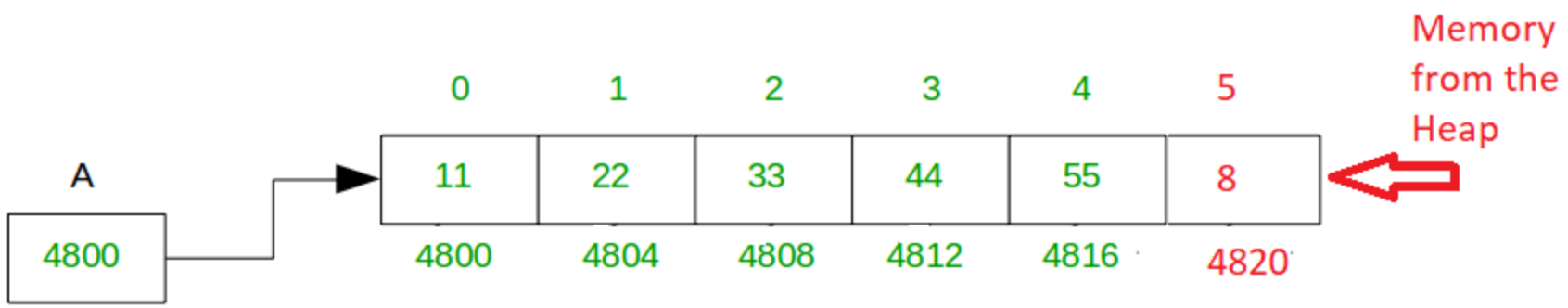


Image Source -
<https://www.geeksforgeeks.org/dsa/how-do-dynamic-arrays-work/>

Arrays in Concept – 2D Arrays

- What do you imagine when you think of 2D arrays?
- We can have an array of arrays to create a 2D array.
- What might this be useful for?

Arrays in Concept – 2D Arrays

[0] [0]	[0] [1]	[0][2]	[0][3]	[0][4]	[0][5]	[0][6]
[1][0]	[1] [1]	[1] [2]	[1] [3]	[1] [4]	[1] [5]	[1] [6]
[2] [0]	[2] [1]	[2] [2]	[2] [3]	[2] [4]	[2] [5]	[2] [6]
[3] [0]	[3] [1]	[3] [2]	[3] [3]	[3] [4]	[3] [5]	[3] [6]
[4] [0]	[4] [1]	[4] [2]	[4] [3]	[4] [4]	[4] [5]	[4] [6]
[5] [0]	[5] [1]	[5] [2]	[5] [3]	[5] [4]	[5] [5]	[5] [6]

Image Source – Ed
Lessons Week 6

Arrays in Concept – 2D Arrays

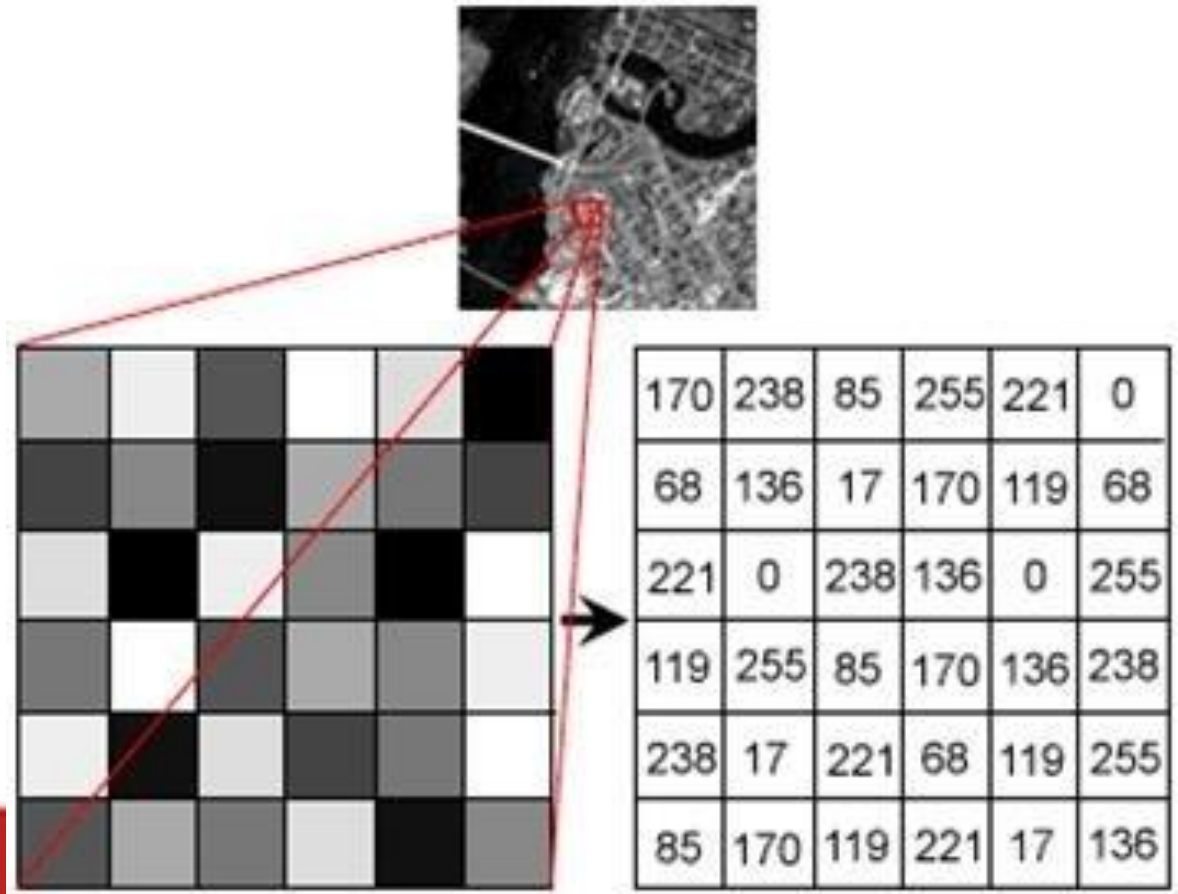


Image Source - Neves, António & Barbosa, Bruno & Soares, Sandra & Dimas, Isabel. (2018). Analysis of Emotions From Body Postures Based on Digital Imaging.

Python Lists

- Lets look at some of the features in Python Lists
- Lists in Python are dynamic and automatically handled by the Python Memory Manager
- Keep in mind in other programming languages arrays you might need to manage the memory allocation of arrays yourself.

Lists are ordered

```
0      1      2      3
['Bob', 'Susie', 'Randy', 'Krog the Destroyer']
10      23      9      34
```

Lists are Mutable (Changeable)

```
listNames[3] = "Krog the Accountant"  
print(listNames)
```

```
['Bob', 'Susie', 'Randy', 'Krog the Accountant']
```

Lists allow duplicates

```
listDupes = ["Dog", "Cat", "Rabbit", "Cat", "Cat"]
```

List Comprehensions

```
1 numbers = list(range(1,15))
2 print(numbers)
3 even_numbers = [x for x in numbers if x % 2 == 0]
4 print(even_numbers)
5 doubled_numbers = [x * 2 for x in numbers]
6 print(doubled_numbers)
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
• (.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009> & C:/Users/De
swin/cos10009/.venv/Scripts/python.exe c:/Users/Default.DESKTOP-8TCQEDR/Documents/ws/
ons.py
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
[2, 4, 6, 8, 10, 12, 14]
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28]
○ (.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009>
```

List Methods

- https://www.w3schools.com/python/python_lists_methods.asp

Method	Description
<u>append()</u>	Adds an element at the end of the list
<u>clear()</u>	Removes all the elements from the list
<u>copy()</u>	Returns a copy of the list
<u>count()</u>	Returns the number of elements with the specified value
<u>extend()</u>	Add the elements of a list (or any iterable), to the end of the current list
<u>index()</u>	Returns the index of the first element with the specified value
<u>insert()</u>	Adds an element at the specified position
<u>pop()</u>	Removes the element at the specified position
<u>remove()</u>	Removes the item with the specified value
<u>reverse()</u>	Reverses the order of the list
<u>sort()</u>	Sorts the list

Searching Arrays - Theory

- When an array has lots of data in it, we need to think about how we might find an item in it?
- What if we want to find a specific user from a list of users?
- Search for a score in a list of scores?
- We need to implement a function to search the list.

Linear Search - Pseudocode

```
procedure linearSearch(list, target)
  n = length of list

  for i from 0 to n - 1
    if list[i] == target
      return i      // return index where found
    end if
  end for

  return -1          // not found
end procedure
```

Linear Search - Implementation

```
def linearSearch(inputList, target):  
    for item in inputList:  
        if item == target:  
            return target  
  
    return - 1
```

What are some problems with this implementation?

```
10 print(linearSearch(ages, 34))
11 print(linearSearch(ages, 99))
12 print("-----")
13 print(linearSearch(names, "Sam"))
14 print(linearSearch(names, "Billy Mac"))
15
```

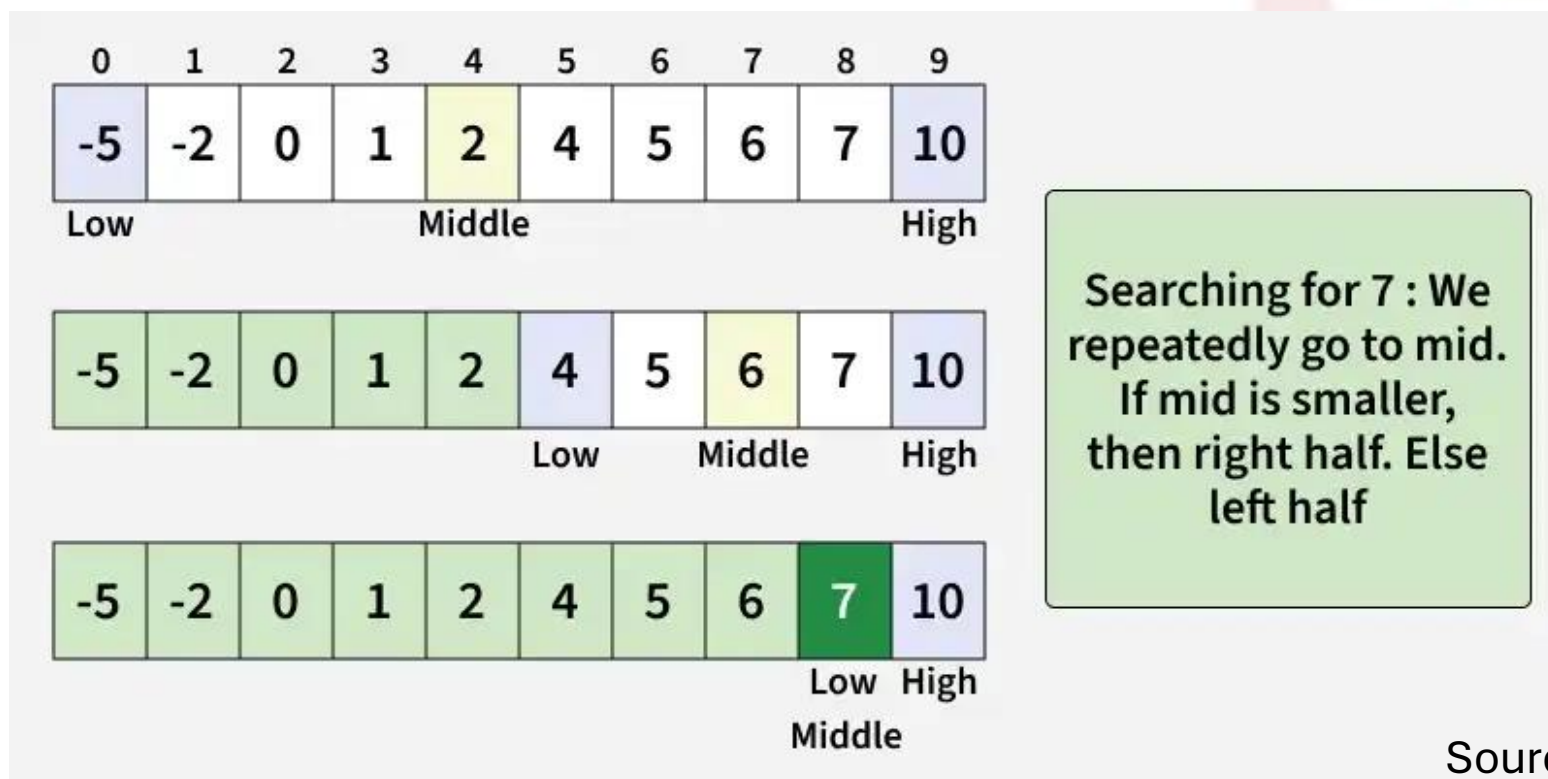
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009> & C:/Users/Default.DESKTOP-8TCQEDR/Documents\Scripts/Activate.ps1
(.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009> & C:/Users/Default.DESKTOP-8TCQEDR/Documents\Scripts/python.exe c:/Users/Default.DESKTOP-8TCQEDR/Documents\ws\swin\cos10009/test/searching.py
34
-1
-----
Sam
-1
(.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009>
```

Binary Search

- I am thinking of a number between 1 and 100. Everytime you guess I will tell you if the number is higher or lower
- Try to guess what I am thinking of?
- How many guesses does it take?

Binary Search - Theory



Source: <https://www.geeksforgeeks.org/dsa/binary-search/>

Binary Search – Bonus

```
procedure binarySearch(list, target)
  low = 0
  high = length of list - 1

  while low <= high
    mid = (low + high) // 2

    if list[mid] == target
      return mid
    else if list[mid] < target
      low = mid + 1
    else
      high = mid - 1
    end if
  end while

  return -1
end procedure
```

Try to implement your own Binary Search algorithm using this pseudocode

Keep in mind the list must be pre-sorted.
We will cover sorting in Week 9 so just use a pre-sorted list for now.

Searching Arrays - Python

Also see: Lecture for W6 and the accompanying code.

```
# Easy python way
print(34 in ages)
print(99 in ages)
print("Sam" in names)
print("Billy Mac" in names)
```

Pointers, stack & heap - Theory

- We're going to look at what happens under the hood of the memory in the computer when a program runs.
- This is very important to know when working with low level languages like C or Rust.
- We will study it anyway because it is useful to know, but the Python Memory Manager will handle it behind the scenes when writing python programs.

Parameter Passing

Parameter Passing

Parameters can be:

- Pass by Value; OR
- Pass by Reference

Pass By Value

- Pass by value takes a copy of the variable and passes it to the called function or procedure:
- The original value is unchanged, a copy is made in the memory.
- Can be slower, have overhead for large copies.

Pass by Reference

- Instead of making a copy, the memory address of the original is passed to the function.
- Changes made to the data change the original. There is no backup.
- Faster for large objects but is less safe.

Python uses Pass by Object Reference

- In Python we pass not by reference or by value.
- In python everything is an object. Objects can be Mutable (changing) or Immutable (unchanging)
- Int, str, float etc are examples of Immutable data.
- Lists, objects are examples of mutable data.

Python Immutable Example

```
1 def change(x):  
2     x = 10  
3  
4 a = 5  
5 change(a)  
6 print(a)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009\.venv\Scripts\Activate.ps1  
(.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009\.venv\Scripts\python.exe c:/Users/Default.DESKTOP-8TCQEDR\Documents\ct_reference.py  
5  
(.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\
```

Python Mutable Example

```
9  def change(Lst):
10      Lst.append(4)
11
12  numbers = [1, 2, 3]
13  change(numbers)
14  print(numbers)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009\.venv\Scripts\python.exe c:/Users/Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009\ct_reference.py
5
(.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009\.venv\Scripts\python.exe c:/Users/Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009\ct_reference.py
[1, 2, 3, 4]
(.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009\.venv\Scripts\python.exe c:/Users/Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009\ct_reference.py

Stack & Heap

Stack

Stores function frames

Very fast

Automatically cleaned when function returns

Structured (LIFO)

Small size

Heap

Stores objects

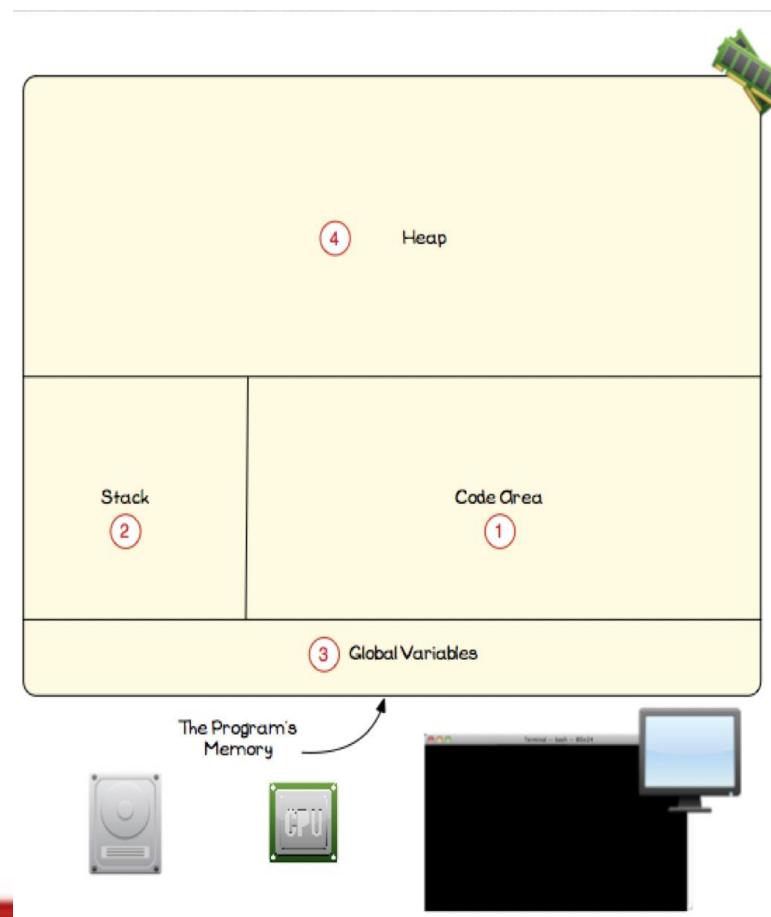
Slower

Cleaned by garbage collector

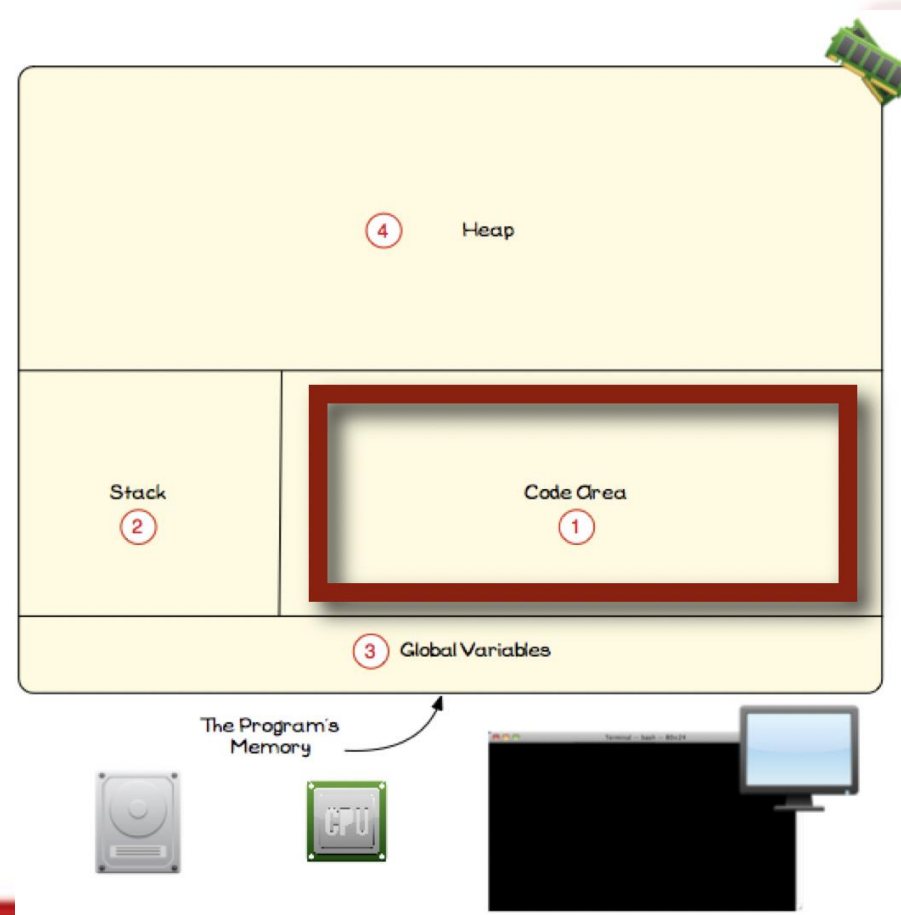
Dynamic allocation

Large size

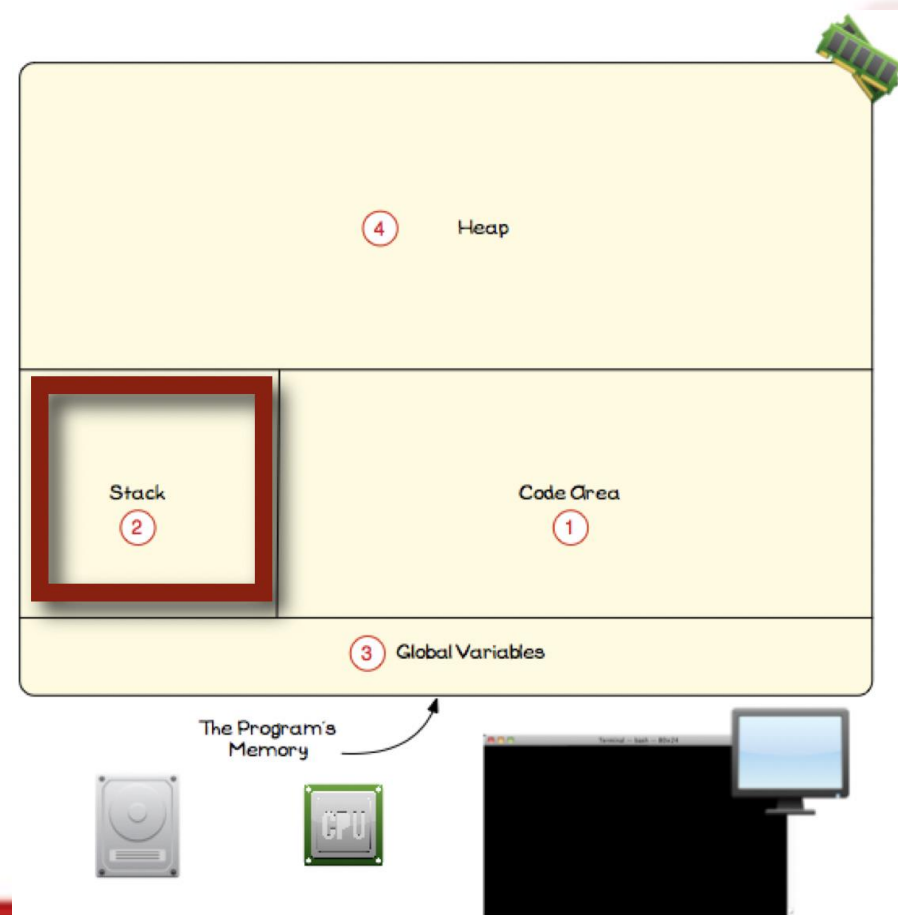
Memory Diagram



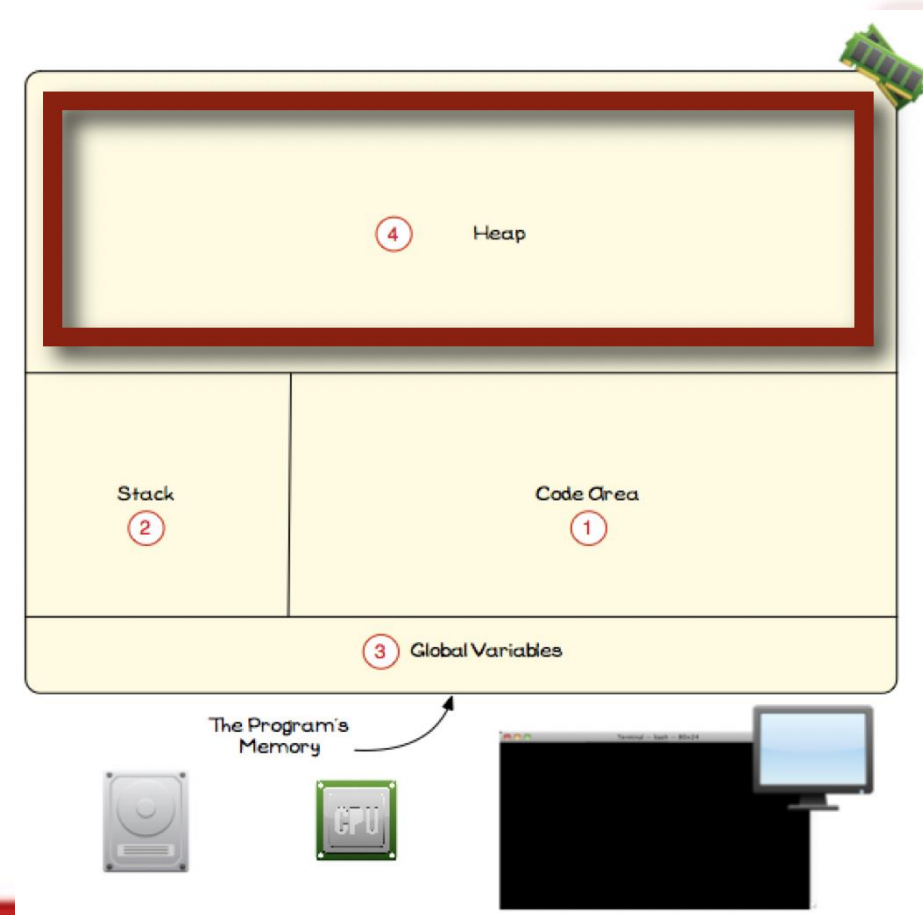
Memory Diagram – Code Area



Memory Diagram - Stack



Memory Diagram - Heap



Garbage Collection - Theory

Garbage collection (GC) is:

- Automatic memory management.

Instead of the programmer manually freeing memory, the runtime detects objects that are no longer reachable and reclaims their memory.

In theory, GC systems answer one question:

- "Is this object still reachable?"

If no references can reach it, it's garbage → delete it.

Garbage Collection Python

Python used a Garbage Collector to free up memory when no references to the data exist. It's automatic and behind the scenes.

You don't need to manually delete a reference to an object unless you want to free up the space. Please note `del` doesn't delete it from memory, it deletes the reference so the GC will clean it up later.

```
2 a = [1, 2, 3]
3 print("Reference count before:", sys.getrefcount(a))
4 b = a # another reference to the same object
5 print("Reference count after assigning b:", sys.getrefcount(a))
6 del b # delete one reference
7 print("Reference count after deleting b:", sys.getrefcount(a))
8
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009> & C:/Users/Default.DESKTOP-8TCQEDR/Documents\swin\cos10009/.venv/Scripts/Activate.ps1
(.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009> & C:/Users/Default.DESKTOP-8TCQEDR/Documents\swin\cos10009/.venv/Scripts/python.exe c:/Users/Default.DESKTOP-8TCQEDR/Documents\ws\swin\cos10009/test/ence.py
Reference count before: 2
Reference count after assigning b: 3
Reference count after deleting b: 2
(.venv) PS C:\Users\Default.DESKTOP-8TCQEDR\Documents\ws\swin\cos10009> & C:/Users/Default.DESKTOP-8TCQEDR/Documents\swin\cos10009/.venv/Scripts/python.exe c:/Users/Default.DESKTOP-8TCQEDR/Documents\ws\swin\cos10009/test/ence.py
```

Custom Project Design

It's time to start thinking about the design of your Custom Program. If you are unsure about what you want to do please reach out.

Week 7: 7.4C M Custom Program Design

 Publish Assign to Edit

In this task you will provide a plan and overview of the structure of a custom program (something you would be interested in creating).

[Credit Task 6.3 - Custom Program Design.docx](#) 

Upcoming Week 9

- Music player submission
- This is a major task, you will have to demonstrate and explain your code in order to pass.
- You MUST use While loops only for this task and will not pass if you use any other kind.

Your Text Based Music Application must have the following functionality:

You must use while loops for this task - do not use `for` loops or other loops.

Problem Scenario - Can you solve it?

You have:

- players = ["Tom", "Sara", "Leo", "Mina"]
scores = [150, 200, 120, 180]
- Tasks:
- Find the highest score
- Print the name of the winner
- Find the rank of a given player

